# Chenchu Raju

# 2303A52206

# AI Assissted Coding

# Lab_9.4

**Task 1:** Auto-Generating Function Documentation in a Shared Codebase

AI.py > multiply

```python
def add(a, b):
    """
    Adds two numbers and returns the result.

    Args:
        a (int | float): The first number.
        b (int | float): The second number.

    Returns:
        int | float: The sum of a and b.

    Example:
        >>> add(5, 3)
        8
    """
    return a + b
def subtract(a, b):
    """
    Subtracts the second number from the first number.

    Args:
        a (int | float): The number to subtract from.
        b (int | float): The number to subtract.

    Returns:
        int | float: The difference between a and b.
    Example:
        >>> subtract(10, 4)
        6
    """
    return a - b
def multiply(a, b):
    """
    Multiplies two numbers.

    Args:
        a (int | float): The first number.
        b (int | float): The second number.

    Returns:
        int | float: The product of a and b.

    Example:
        >>> multiply(6, 7)
        42
    """
    return a * b


def divide(a, b):
    """
    Divides the first number by the second number.
```

```python
    Divides the first number by the second number.

    Args:
        a (int | float): The numerator.
        b (int | float): The denominator.

    Returns:
        float: The result of division.

    Raises:
        ValueError: If b is zero.

    Example:
        >>> divide(10, 2)
        5.0
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b


def is_even(number):
    """
    Checks whether a number is even.
```

```python

def factorial(n):
    """
    Calculates the factorial of a non-negative integer.

    Args:
        n (int): A non-negative integer.

    Returns:
        int: The factorial of n.

    Raises:
        ValueError: If n is negative.

    Example:
        >>> factorial(5)
        120
    """
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
print(add(5, 3))        # Output: 8
print(subtract(10, 4))  # Output: 6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
\Desktop\Desktop\AI2\AI.py'
8
6
```

**Summary:** This section defines fundamental mathematical functions such as addition, subtraction, multiplication, and division, each with proper docstrings explaining usage, arguments, and return values. It also includes functions to check whether a number is even and to calculate the factorial of a number with error handling for invalid input. These functions demonstrate clean coding practices, input validation, and reusable utility design. Example print statements show how to use these functions. Overall, this section focuses on basic arithmetic operations and number-related utilities.

**Task 2:** Enhancing Readability Through AI-Generated Inline Comments

AI.py > ...

```python
def fibonacci(n):
    if n <= 0:
        return []
    if n == 1:
        return [0]

    sequence = [0, 1]

    for i in range(2, n):
        # Each number is the sum of the previous two numbers
        sequence.append(sequence[i - 1] + sequence[i - 2])

    return sequence


def binary_search(arr, target):
    left = 0
    right = len(arr) - 1

    while left <= right:
        # Prevents overflow and ensures correct mid calculation
        mid = left + (right - left) // 2
```

```python
def binary_search(arr, target):
    left = 0
    right = len(arr) - 1

    while left <= right:
        # Prevents overflow and ensures correct mid calculation
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid

        elif arr[mid] < target:
            # Target must be in the right half (array assumed sorted)
            left = mid + 1
        else:
            # Target must be in the left half
            right = mid - 1

    return -1


def bubble_sort(arr):
    n = len(arr)
```

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False  # Tracks whether a swap happened in this pass

        for j in range(0, n - i - 1):
            # Largest element in this pass moves to its correct position at the end
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Optimization: Stop early if the array is already sorted
        if not swapped:
            break

    return arr


if __name__ == "__main__":
    print("Fibonacci:", fibonacci(7))
```

Welcome  AI.py

AI.py > ...

```python
def bubble_sort(arr):
    return arr


if __name__ == "__main__":
    print("Fibonacci:", fibonacci(7))

    sorted_list = bubble_sort([64, 34, 25, 12, 22, 11, 90])
    print("Sorted:", sorted_list)

    print("Index of 25:", binary_search(sorted_list, 25))
```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

```
Fibonacci: [0, 1, 1, 2, 3, 5, 8]
Sorted: [11, 12, 22, 25, 34, 64, 90]
Index of 25: 3
```

**Summary:** This section implements three important algorithms: Fibonacci sequence generation, binary search, and bubble sort. The Fibonacci function generates a sequence where each number is the sum of the previous two numbers. The binary search function efficiently finds a target value in a sorted array using a divide-and-conquer approach. The bubble sort function sorts an array by repeatedly swapping adjacent elements and includes an optimization to stop early if already sorted. This section demonstrates algorithm design, loops, conditionals, and performance optimization techniques

**Task 3:** Generating Module-Level Documentation for a Python Package

Welcome  AI.py

AI.py > ...

```python
"""
student_utils.py

This module provides utility classes and functions for managing
student academic records. It includes functionality for storing
student data, calculating averages, assigning grades, and managing
multiple student records.

Dependencies:
    - No external libraries required (built-in Python only)

Key Components:
    - Student: Represents an individual student and their marks.
    - StudentManager: Manages a collection of Student objects.
    - calculate_grade(): Determines grade based on average score.

Example:
    >>> from student_utils import Student, StudentManager
    >>> s = Student("Vikas", 101, [85, 90, 88])
    >>> s.calculate_average()
    87.67
```

```python
    >>> s.calculate_average()
    87.67
    >>> manager = StudentManager()
    >>> manager.add_student(s)
"""

class Student:
    def __init__(self, name, roll_number, marks=None):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks if marks else []

    def calculate_average(self):
        if not self.marks:
            return 0.0
        return sum(self.marks) / len(self.marks)


def calculate_grade(avg):
    if avg >= 90:
        return "A"
    elif avg >= 80:
```

```python
def calculate_grade(avg):
    elif avg >= 80:
        return "B"
    elif avg >= 70:
        return "C"
    elif avg >= 60:
        return "D"
    return "F"


class StudentManager:
    def __init__(self):
        self.students = []

    def add_student(self, student):
        self.students.append(student)
print("Student Manager initialized.")
print("Adding student Raju with roll number 101 and marks [85, 90, 88].")
Raju = Student("Raju", 101, [85, 90, 88])
manager = StudentManager()
manager.add_student(Raju)
print(f"Student {Raju.name} added wi erage marks: {Raju.calculate_average():.2f}
```

Ports

```
Student Manager initialized.
Adding student Raju with roll number 101 and marks [85, 90, 88].
Index of 25: 3
Student Manager initialized.
Adding student Raju with roll number 101 and marks [85, 90, 88].
Student Manager initialized.
Adding student Raju with roll number 101 and marks [85, 90, 88].
Adding student Raju with roll number 101 and marks [85, 90, 88].
Student Raju added with average marks: 87.67 and grade: B
```

Task 4: Converting Developer Comments into Structured Docstrings

Welcome AI.py

AI.py > ...

```python
def factorial(n):
    """
    Calculate the factorial of a non-negative integer.

    Args:
        n (int): A non-negative integer.

    Returns:
        int: The factorial of the given number.

    Raises:
        ValueError: If n is negative.

    Example:
        >>> factorial(5)
        120
    """
    if n < 0:
        raise ValueError("Negative numbers are not allowed.")

    result = 1
    for i in range(1, n + 1):
        result *= i
```

Ports

```python
def is_prime(num):
    """
    Determine whether a number is prime.

    Args:
        num (int): The number to check.

    Returns:
        bool: True if the number is prime, False otherwise.

    Example:
        >>> is_prime(7)
        True
    """
    if num <= 1:
        return False

    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False

    return True
```

```
AI.py > is_prime
def is_prime(num):
            return False

        return True
print("Factorial of 5:", factorial(5))  # Output: 120
print("Is 7 prime?", is_prime(7))        # Output: True
```

```
Factorial of 5: 120
Factorial of 5: 120
Is 7 prime? True
```

Summary: This section implements an object-oriented student record management system using classes. The Student class stores student details such as name, roll number, and marks, and calculates the average score. The calculate_grade function assigns grades based on average marks using conditional logic. The StudentManager class manages multiple student objects by storing and organizing student records. This section demonstrates object-oriented programming concepts such as classes, objects, methods, and data management.

Task 5: Building a Mini Automatic Documentation Generator

```
Welcome    AI.py
AI.py > is_prime
import ast
import os

def insert_docstrings(file_path):
    if not os.path.exists(file_path):
        print(f"Error: File '{file_path}' not found.")
        return

    with open(file_path, "r") as f:
        lines = f.readlines()

    tree = ast.parse("".join(lines))
    output_lines = lines[:]

    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            line_no = node.lineno
            doc = f'''    """
    {node.name} function.

    Args:
        TODO: Add parameter descriptions.
```

```python
def insert_docstrings(file_path):
                line_no = node.lineno
                doc = f'''    """
        {node.name} function.

        Args:
            TODO: Add parameter descriptions.

        Returns:
            TODO: Add return description.
        """
    '''
                output_lines.insert(line_no, doc)

        new_file = "documented_" + os.path.basename(file_path)

        with open(new_file, "w") as f:
            f.writelines(output_lines)

        print(f"Documentation generated in {new_file}")
    print("Inserting docstrings into 'week9.4.py'...")
    insert_docstrings("week9.4.py")
```

```
Inserting docstrings into 'week9.4.py'...
Is 7 prime? True
Inserting docstrings into 'week9.4.py'...
Inserting docstrings into 'week9.4.py'...
Error: File 'week9.4.py' not found.
PS C:\Users\Admin\OneDrive\Documents\Desktop\Desktop\AI2>
```

Summary: This section creates a tool that automatically inserts documentation (docstrings) into Python functions using the ast module. It reads a Python file, analyzes its structure to find function definitions, and inserts a template docstring for each function. The script then saves the updated version into a new file with documentation added. It also includes file existence checking and user feedback messages. This section demonstrates automation, file handling, abstract syntax tree processing, and improving code maintainability.